```
// Headless verification of Demo 28's chain math (Theorem 3)
const Zv=150, Zh=250, D=Zh-Zv;
const toScreen=(X,Y,Z)=>[-X*Zv/(Z-Zv), -Y*Zh/(Z-Zh)];
function chainCoeffs(E1,E2){
  const Dx=E2[0]-E1[0], Dy=E2[1]-E1[1], Dz=E2[2]-E1[2];
  const p1=-Dx*Zv, p0=-E1[0]*Zv, q1=Dz, q0=E1[2]-Zv;
  const r1=-Dy*Zh, r0=-E1[1]*Zh, s1=Dz, s0=E1[2]-Zh;
  return {a:r1*q0-r0*q1, b:r0*p1-r1*p0, c:s1*q0-s0*q1, d:s0*p1-s1*p0};
}
const mob=(M,x)=>(M.a*x+M.b)/(M.c*x+M.d);

// 1) exact coefficients: every image point of a random line satisfies y = mob(x)
let w=0;
for(let trial=0; trial<3000; trial++){
  const E1=[(Math.random()-0.5)*240,(Math.random()-0.5)*240,270+Math.random()*170
  const E2=[(Math.random()-0.5)*240,(Math.random()-0.5)*240,270+Math.random()*170
  if(Math.abs(E2[2]-E1[2])<2) continue;
  const M=chainCoeffs(E1,E2);
  const sc=Math.max(Math.abs(M.a),Math.abs(M.b),Math.abs(M.c),Math.abs(M.d));
  for(let i=0;i<=20;i++){
    const u=-1+3*i/20;
    const Z=E1[2]+u*(E2[2]-E1[2]);
    if(Math.abs(Z-Zv)<2||Math.abs(Z-Zh)<2) continue;
    const [x,y]=toScreen(E1[0]+u*(E2[0]-E1[0]), E1[1]+u*(E2[1]-E1[1]), Z);
    if(Math.abs(M.c*x+M.d)/sc < 1e-6) continue;
    w=Math.max(w, Math.abs(mob(M,x)-y));
  }
}
console.log("chain identity worst residual   :", w.toExponential(2), "(doc: 2e-12

// 2) case 2: c = Dz*(Zh-Zv) exactly -> straight iff Dz=0
const E1=[10,20,300], E2=[80,-40,300];
const M2=chainCoeffs(E1,E2);
console.log("Dz=0 gives c =", M2.c, " (affine chain: straight)");

// 3) asymptote identities: x* = screen-x where m crosses z=Zh, y* likewise at Zv
const A=[-70,-20,310], B=[80,30,420];
const M=chainCoeffs(A,B);
const Dx=B[0]-A[0], Dy=B[1]-A[1], Dz=B[2]-A[2];
const uh=(Zh-A[2])/Dz, uv=(Zv-A[2])/Dz;
const xCross = -(A[0]+uh*Dx)*Zv/(Zh-Zv);
const yCross = -(A[1]+uv*Dy)*Zh/(Zv-Zh);
console.log("x* vs crossing-x:", (-M.d/M.c).toFixed(9), xCross.toFixed(9));
console.log("y* vs crossing-y:", ( M.a/M.c).toFixed(9), yCross.toFixed(9));
```

```
// 4) case 3 (the cross): translate m onto l1; all other points share one t,
//    and the collapsed trace line equals the limiting vertical asymptote
function closest(P1,u,P2,v){
  const w0=[P1[0]-P2[0],P1[1]-P2[1],P1[2]-P2[2]];
  const aa=u[0]**2+u[1]**2+u[2]**2, bb=u[0]*v[0]+u[1]*v[1]+u[2]*v[2];
  const cc=v[0]**2+v[1]**2+v[2]**2, dd=u[0]*w0[0]+u[1]*w0[1]+u[2]*w0[2];
  const ee=v[0]*w0[0]+v[1]*w0[1]+v[2]*w0[2], den=aa*cc-bb*bb;
  const sc=(bb*ee-cc*dd)/den, tc=(aa*ee-bb*dd)/den;
  return {pa:[P1[0]+sc*u[0],P1[1]+sc*u[1],P1[2]+sc*u[2]],
          pb:[P2[0]+tc*v[0],P2[1]+tc*v[1],P2[2]+tc*v[2]]};
}
const u=[Dx,Dy,Dz];
const cl=closest(A,u,[0,0,Zv],[0,1,0]);
const dv=[cl.pb[0]-cl.pa[0],cl.pb[1]-cl.pa[1],cl.pb[2]-cl.pa[2]];
const A2=[A[0]+dv[0],A[1]+dv[1],A[2]+dv[2]];
// t(u) for points of the touched line:
const tOf=(uu)=>{const X=A2[0]+uu*Dx, Z=A2[2]+uu*Dz; return X*D/(Z-Zv);};
let tmin=1e18, tmax=-1e18;
for(let i=1;i<=30;i++){const uu=(Zv-A2[2])/Dz + i*0.1; const t=tOf(uu);
  tmin=Math.min(tmin,t); tmax=Math.max(tmax,t);}
console.log("t constant along touched line   :", tmin.toFixed(9), "==", tmax.toFi
console.log("collapsed trace x = -1.5*t0     :", (-1.5*tmin).toFixed(6),
            " vs -Zv*Dx/Dz =", (-Zv*Dx/Dz).toFixed(6));
```